

ABAP Development and Naming Standards

Table of Contents

1.	OVERVIEW.....	4
2.	GETTING STARTED.....	4
2.1	PROGRAM ORGANIZATION.....	4
2.1.1	One Command Per Line.....	4
2.1.2	Report/Program Statement.....	4
2.1.3	ABAP Program Header.....	4
2.1.4	ABAP Modification Log.....	5
2.1.5	Tables Statement.....	5
2.1.6	Global Variables.....	6
2.1.7	Selection-Screen.....	6
2.2	PROGRAM NAMING STANDARDS.....	6
2.2.1	Tabl 1 Functional Process.....	6
2.2.2	Table 2 Functions.....	7
2.2.3	Proposal for Development Classes (Package).....	7
2.3	VARIABLE NAMING CONVENTIONS.....	8
2.3.1	Table 3 Variable Types.....	8
2.4	COMMENTS IN ABAP PROGRAMS.....	9
2.4.1	For declarations: -.....	10
2.4.2	Events:-.....	10
	SHOULD BE OF THE TYPE GIVEN BELOW.....	10
2.4.3	Subroutines:-.....	10
2.5	TYPES, INTERNAL TABLES AND WORKAREAS.....	11
	KEY GUIDELINES.....	11
3.1	TECHNIQUES NEVER TO BE USED:.....	11
3.1.1	Direct database updates of SAP standard tables.....	11
3.1.2	User-IDs in programs.....	12
3.1.3	Usage Function Modules that are not released.....	12
3.1.4	Authorization Group / Activity Group.....	12
3.1.5	Create new Message Class.....	12
3.1.6	Hard coding of constant.....	12
3.1.7	Editor Locks.....	12
3.1.8	Password Authorization:.....	12
3.1.9	Secure Store and Forward Mechanism (SSF):.....	12
3.1.10	Audit Logs.....	12
3.1.11	Web Dynpro & ABAP.....	13
3.1.12	Get method.....	13
3.1.13	Decoding.....	13
3.1.14	Cookies.....	13
3.1.15	SQL Checks.....	13

ABAP Development and Naming Standards	ProcessWeaver
3.2 TECHNIQUES THAT REQUIRE PRIOR APPROVAL.....	13
3.2.1 Create new Function Group.....	13
3.2.2 Import / Export to Memory.....	13
3.2.3 Common Part.....	14
3.2.4 Dynamic Form Calls.....	14
3.2.5 SQL.....	14
3.2.6 File Management.....	14
3.3 TECHNIQUES TO USE SPARINGLY.....	14
3.3.1 Field Symbols.....	14
3.3.2 Get/Set.....	14
3.3.3 Field Groups.....	14
3.4 SMARTFORMS Vs SAPSCRIPTS.....	14
3.5 ABAP LIST VIEWER (ALV).....	14
4. ERROR HANDLING.....	15
4.1. ALWAYS TEST SY-SUBRC.....	15
4.2. NOTIFICATION VIA WORKFLOW :.....	15
4.3. STANDARD ERROR REPORTING IN BDC AND CALL TRANSACTION.....	15
4.4. BAPI ERRORS.....	15
4.5. FUNCTION MODULES.....	15
5. MODULARIZATION.....	15
5.1. FORMS.....	16
5.1.1. Parameter Passing in Forms.....	16
5.2 DIALOG MODULES.....	17
5.3 INCLUDE PROGRAMS.....	17
5.4 FUNCTION MODULES.....	17
5.4.1 Error Handling.....	17
5.4.2 Release status of Function Modules.....	18
5.5 USER EXITS.....	18
6. PERFORMANCE.....	19
6.1 DATABASE ACCESS.....	19
6.1.1 Master Data and Configuration Data.....	20
6.1.2 SELECT and SELECT SINGLE.....	20
6.1.3 Nested SELECTs versus table views or JOINS.....	20
6.1.4 Row-Level Processing of a table.....	20
6.1.5 SELECT * versus Selecting individual fields.....	20
6.1.6 FOR ALL ENTRIES IN.....	20
6.2 INTERNAL TABLES.....	21
6.2.1 Reading single records of internal tables.....	21
6.2.2 The SORT statement.....	21
6.2.3 Small Internal tables Vs. Complete Internal Tables.....	21
6.3 Logic Structures.....	21
6.3.1 Decision trees.....	21
6.3.2 CASE vs. Nested IFs.....	21
6.4 FIELDS, STRUCTURES ETC.....	21
6.4.1 Field assignment.....	21
6.4.2 Field conversion.....	21
6.4.3 Field includes.....	21
6.5 GENERAL CODING.....	22
6.5.1 Subroutine Usage.....	22
6.5.2 Dead Code.....	22

ABAP Development and Naming Standards	ProcessWeaver
7. GENERAL STANDARDS.....	22
7.1 PRETTY PRINTER.....	22
7.2 CHECK.....	22
7.3 SY- FIELDS AND SELECTION-SCREEN PARAMETERS.....	22
7.4 FIELD DATA VALIDATION.....	22
7.4.1 <i>Selection Screen Parameters</i>	22
7.4.2 <i>Input Data Errors</i>	23
7.4.3 <i>Input Validation</i>	23
7.5 TEXT HANDLING.....	23
7.6 MESSAGES.....	23
EXAMPLES:.....	24
7.7 'EXTENDED PROGRAM CHECK' FOR ABAP PROGRAMS.....	24
7.7.1 <i>Using the checks</i>	24
7.7.2 <i>Extended Checks in ABAP Code Review</i>	24
7.8 REPORT_FORMATTING.....	25
7.9 BATCH DATA INPUT.....	25
7.10 LOCK OBJECTS.....	25
7.11 DEBUGGING.....	25
8 SECURITY STANDARDS.....	25
8 ADDITIONAL NAMING STANDARDS.....	25
8.1 OTHER PROGRAM OBJECTS.....	25
8.1.1 <i>ABAP Module Pools</i>	25
8.1.2 <i>Temporary ABAP objects</i>	25
8.1.3 <i>Program Attributes</i>	25
8.2 OBJECT ORIENTED OBJECTS.....	26
8.2.1 <i>Classes and Interfaces</i>	26
8.3 CORRECTIONS/TRANSPORTS PROCESS.....	26
8.3.1 <i>Text on Corrections/Transports</i>	26
8.4 DATA DICTIONARY OBJECTS.....	26
8.4.1 <i>Tables, Table Types, Structures and Views</i>	26
8.4.2 <i>Type Groups & Type Pools</i>	27
8.4.3 <i>Search Help and Matchcode Objects</i>	27
8.4.4 <i>Lock Objects</i>	27
8.5 PROGRAM VARIANTS.....	27
8.6 TRANSACTION CODES.....	27
8.7 SAP SCRIPT OBJECTS.....	28
8.7.1 <i>Layout Sets (Forms)</i>	28
8.7.2 <i>Style</i>	28
8.7.3 <i>Paragraphs and Character Formats</i>	28
8.8 IDOC TYPES.....	28
8.9 STANDARD TEXT ID'S & STANDARD TEXT NAMES.....	28
8.10 USER EXITS.....	28

1. OVERVIEW

This document is intended to provide ABAP Coding Standards, Naming Standards, and optimization hints to ABAP programmers for the SAP implementation. Adherence to these standards will allow for easy maintenance, tracking of modifications and a high level of documentation in the programs that are created to enhance the SAP applications. Adherence to certain standards is required to pass code review.

A further general guideline is to modularize your code as far as possible, without going to extremes. This also makes programs easier to understand and modify.

The Naming Standards are used for better object administration and faster location of objects.

Note: NO MOD is allowed and it will only through approved/released OSS note. Additionally, need approval from GAP Manager and/or PMO.

2. Getting Started

2.1 Program Organization

2.1.1 One Command Per Line

Start each new command on a new line. This will allow for easier deleting, commenting, and debugging.

2.1.2 Report/Program Statement

Start every report with REPORT *program name* NO STANDARD PAGE HEADING LINE SIZE xxx MESSAGE ID xx.

2.1.3 ABAP Program Header

The following data must be used for each ABAP program header (immediately after the REPORT statement):

.	
Author:	Name of programmer
Date:	Month & year program is first written
Short Program Description:	Approximately 4 to 20 sentences.
Object Id	Process Weaver generated Id of the object with description of the object
Program Objective	Updates Table () Downloads Data () Outputs List () Calls Layout ()
Includes	If the program has any includes list the include programs
Copied From	If some other program is copied give the name of the program Remarks

Give specific information if any

Development Request Number

Also see *Appendix H* for a program template/shell.

2.1.4 ABAP Modification Log

The ABAP Modification Log should be immediately after the ABAP Program Header. For programs that were already transported to the Production system, all modifications must be documented in the Modification Log in the program header, with the latest change added to the top of the Log.

A comment that includes the Modification Identification must appear in each line of code that was modified, including deleted lines, which should be commented out and never physically removed. Hence the Modification Identification can always be used as the search string. The SAP Workbench Organizer generates the Transport Number when you make your first save from the ABAP editor.

Example:

```
*-----*
Modification Log
*-----*
Mod Tag EID   Date   Transport#  Short description
MOD10  E45011 09/21/10 DEVK913831  Add Jest Table
```

In addition to the modification log, maintain the detailed description in the transport documentation tab.

In addition maintain the detailed description in the Transport documentation tab

```
*-----*
Modification Log
*-----*

Modification Identification MOD01
Modification Date 17th April 2000
Modification Reference Reference Document for
Modification (FS Version 09/
HAR30001_IL_INF00092)
Modification Details Give the Description of the
change
Developer Surya (Name of the Developer)
Transport Request Number ARRK901111
*-----*
```

The most recent change should be on top – so you can see it on the same screen as the Program Header information.

2.1.5 Tables Statement

TABLE's statements should be written for all tables used and should be immediately after the Modification Log.

2.1.6 Global Variables

All Global Variables should be defined at the top of the program, after the Tables statement.

Use "LIKE" to declare a variable instead of specifying the length and type separately, wherever applicable.

2.1.7 Selection-Screen

All Selection-Screen commands should be in one place – after the Global Variables.

2.2 Program Naming Standards

The First Nine characters are the default naming space maintained by Process Weaver is /pweaver/. The next three characters will be the product name of Process Weaver like ECS, EFA and so on. We have reserved the first thirteen characters as mandatory in the format listed below. Note that all the production programs should start with a "/pweaver/". For all test and/or \$TMP programs or OSS related programs, we reserve the first character "Z". The custom transaction code naming will be "/pweaver/<3char Product name>****" any meaningful character/letters. Please DO NOT use underscore in Tcodes.

Position 1-9	/pweaver/
Position 10-12	Product Name see 2.2.1 Tabl 1 page 6
Position 13	_ (underscore)
Position 14-40	Descriptive Sequential numbering is permitted only if it follows at least three characters of description or abbreviation.

2.2.1 Tabl 1 Functional Process.

Product Name	Position 10 – 12	SAP Module
Enterprise Centralized Shipping	ECS	ECS
Enterprise Freight Audit	EFA	EFA
Enterprise Freight Shopping	EFS	EFS
Enterprise Address Validation	EAV	EAV
Enterprise Track and Trace	ETT	ETT
Business Partner Screening	BPS	BPS
Finance / Costing	FI	FI / CO
Order to Customer	OC	SD
Requirement to Product	RP	MM
Idea to capable Product	IP	PS
Requirement to Customer	RC	MM
Aftermarket / Repair & Overhaul	AF	SM
Project System / Engineering	PS	PS
Manufacturing	MF	PP
Cross Application	CA	CA
Taxware	TX	External system
Human Resource	HR	HR
Plant Maintenance	PM	PM
Data Integrator	DI	External system
Inspector Workbench	IW	IS-AD Inspector Workbench
Warranty	WT	IS-AD Warranty
Government Property	GP	IS-AD Government Property
Basis component	BC	SAP Basis
Quality Management	QM	QM
Business Warehouse	BW	BW

ABAP Development and Naming Standards

ProcessWeaver

Advanced Planner/Optimizer	AP	APO
----------------------------	----	-----

2.2.2 Table 2 Functions

Code	Function
I	Interface (files)
E	Interface (EDI)
B	Interface (BAPI)
/pweaver/MMM_XXXXX	Reporting
/pweaver/MMM_XXXXX	Dialogue programming
/pweaver/MMM_XXXXX	Function Module/Group
/pweaver/MMM_XXXXX_TOP	Include program
/pweaver/XXXXX	Table
/pweaver/ST_XXXXX	Structure
/pweaver/VW_XXXXX	View
/pweaver/SH_XXXXX	Search Help
/pweaver/SC_XXXXX	Scripts
/pweaver/SF_XXXXX	Smart Forms
/pweaver/SS_XXXXX	Smart Styles
/pweaver/ST_XXXXX	Standard Text
/pweaver/TM_XXXXX	Text Modules
	OO Objects:
/pweaver/CL_XXXXX	Class
/pweaver/CA_XXXXX	Class Agent
/pweaver/IF_XXXXX	Interface

Note: Here MMM stands for the Product Name, where as XXXXX stands for the Abap Object Description.

The same naming convention (described above for **Programs**) should be used for **Tables**, **Table Types**, and **Structures**. Very similar naming conventions should be used for **Function Groups**, **Function Modules**, **Classes** and **Interfaces**.

2.2.3 Proposal for Development Classes (Package)

2.2.3.1 Functionality of development classes

All Repository objects in the SAP System are classified according to development classes, i.e. objects, such as ABAP programs, Data Dictionary objects, SAP Script Forms, etc... forming a logical or functional unit, are grouped together in a development class.

The Object Navigator of the ABAP Workbench (Transaction SE80) allows you to browse the objects by development class. To improve clarity when objects are displayed in this way, SAP advises you not to let the number of objects in a development class increase too much, but to distribute the objects to new development classes according to appropriate criteria. On the other hand, too many custom development classes make it more complicated for the ABAP developers and administrators to manage their objects.

Development classes also make it easier to manage corrections and transports, since a development class binds mutually dependent objects together, so they can be corrected and transported together as a unit. A transport layer is assigned to each development

Updated On

-

Proprietary & Confidential

Page 7 of 28

Created On

-

ABAP Development and Naming Standards

ProcessWeaver

class and thus to all objects in that class. The transport layer determines in which SAP System developments or changes to Repository objects are made and to which other systems the objects will be transported once development work has been completed.

When development work starts, the development classes to which the new objects are to be assigned must be in place. All newly created objects in the ABAP Workbench will need to be assigned to a development class.

The development classes are themselves objects in the ABAP Workbench. Changes to existing and newly created development classes are recorded by the Transport Organizer and can be transported into other SAP Systems. The development class of a development class is always itself.

The assignment of an object to a development class is recorded in the Object Directory.

2.2.3.2 Development Class or Package

For the SAP Custom development objects that will be common across Process Weaver, we propose to use the following development classes. For the future SBU / SBE implementations, more development classes can be added.

Development name	class	Team	SAP Modules
/PWEAVER/	ECS	Enterprise Centralized Shipping	ECS
/PWEAVER/	EFA	Enterprise Freight Audit	EFA
/PWEAVER/	EFS	Enterprise Freight Shopping	EFS
/PWEAVER/	EAV	Enterprise Address Validation	EAV
/PWEAVER/	ETT	Enterprise Track and Trace	ETT
/PWEAVER/	BPS	Business Partner Screening	BPS

2.3 Variable Naming Conventions

All variables used in programs, function modules, methods should use the prefixes listed in the following table:

2.3.1 Table 3 Variable Types

Variable Type	Prefix
Global Data:	
Global variables	GV_
Constants	C_
Types	TY_
Work-areas and Structures (global)	GS_
Internal tables (global)	GT_
Selection screen parameter	P_
Select-options	S_
Ranges	R_
Check Box	CH_
List Box	LB_
Radio Button	RB_
Field Groups	FG_

ABAP Development and Naming Standards	ProcessWeaver
---------------------------------------	---------------

Field Symbols	FS_
Local Data:	
Form parameter (using / changing)	CP_
Local Constants	LC_
Local variables	LV_
Work Area (local)	LS_
Internal tables (local)	LT_
Function Module Data:	
FM Import	IM_
FM Export	EX_
FM Changing	CP_
FM Table	GT_
Methods Data:	
Method Importing	MI_
Method Exporting	ME_
Method Changing	MC_
Method Returning	MR_
Flags	FL_
Variants	ZV_
Table Control	TC_
Tab Strip	TS_

Since SAP segment/table-field names are hyphenated with a '-' (dash), use '_' (underline) to separate the words for program-specific variables.

Include Structures: If a structure is available use the same to declare the internal table or Types, instead of giving all the fields in the declaration.

2.4 **Comments in ABAP programs**

Internal commenting of all ABAP code is essential for program maintenance and is required.

Code lines that are modified should be individually labeled using a "(double quote) followed by the Transport number. Extensive blocks of modified code can be identified with 'Begin of...' and 'End of....' comments that contain the Transport number.

Always include comments in the code, especially where the complexity of the program logic is at the higher end of the scale. Comments should explain what the code is doing, not how the code is doing it. Comments should be concise, complete and up-to-date.

Explain the purpose and overall design at the top of every program. Comment all subroutines with their purpose.

2.4.1 **For declarations: -**

Database Tables: -

Description of the table should be documented against the table name

* Declaration for Tables

Global variables:-

Short description of the variable should be documented against the variable name

* Declaration for Variables

```
*-----*
Constants:-
Short description of the constant should be documented
against the constant name
*-----*
* Declaration for Constants
*-----*
Flags:-
Short description of the flag should be documented against
the flag name
*-----*
* Declaration for Flags
*-----*
Internal tables:-
Short description for the use of internal should be documented against the
internal table name
*-----*
* Declaration for Internal tables
*-----*
*-----*
Declaration of includes
*-----*
```

2.4.2 Events:-

Should be of the type given below

Start of Selection:-

```
*-----*
* Start of Selection
*-----*
```

2.4.3 Subroutines:-

```
*&-----*
*& Form test_for_material_type
*&-----*
*          Description
*-----*
* --> Formal parameter type and description
*-----*
```

2.5 Types, Internal Tables and Work areas

Use **TYPE** statements to define data structures and internal tables.

Example:

```
TYPES: BEGIN OF TY_abc,
       bukrs LIKE bkpj-bukrs,
       belnr LIKE bkpj-belnr,
       gjahr LIKE bkpj-gjahr,
       END OF TY_abc.
```

* Explicit Workarea defined; Table without header line:

DATA:

```
LT_abc TYPE TY_abc occurs 0,
LS_abc TYPE TY_abc.
```

* Table with header line:

DATA:

LT_abc TYPE TY_abc occurs 0 with header line.

R/3 releases 4.6 and later contain capabilities for hashing and sorting internal tables that improve system performance. The new syntax should be used whenever coding internal tables. The following examples illustrate the use of this syntax add the syntax for the tables

Example:

LT_vendor type sorted table of LT_vendor_data
with unique key lifnr.
with header line.

Key Guidelines

3.1 *Techniques never to be used:*

If you need to have one of the objects in this list created, ask the Development Manager for approval.

3.1.1 **Direct database updates of SAP standard tables**

Under no circumstances should any program directly update SAP-delivered tables using the INSERT, UPDATE, or DELETE commands. SAP-delivered tables begin with all letters other than /PWEAVER/, Y and Z, and they should only be updated using an SAP transaction. To automate updates to SAP tables via a program, you may only use an available, SAP-delivered BAPI call, or the CALL TRANSACTION command.

The only exception to this is the TVARVC table – used for variant parameters. To modify or add data to TVARVC from a program, prior approval of the Development Manager is required.

3.1.2 **User-IDs in programs**

In no case should a production program or function contain a User-ID as either literal or constant data. In the development system it may be necessary to code temporary breakpoints for specific User-IDs; however, these debugging techniques must be removed before the program will be transported.

3.1.3 **Usage Function Modules that are not released**

Never use Function Modules that are not released, in custom programs.

3.1.4 **Authorization Group / Activity Group**

The Security Lead creates authorization Groups and Activity Groups. Need to use appropriate “Authorization group” for the custom table, if this is updated by a transaction code.

3.1.5 **Create new Message Class**

Creating new message class is NOT allowed. The guideline would be use the standard SAP message class or use the custom message class starting from /PWEAVER/MC_XXXX.

3.1.1 Hard coding of constant

Avoid hard coding constant values which pertains to configuration tables viz. company code, plant code etc.

3.1.7 Editor Locks

Never place editor locks on any of the development objects. A transport request is good enough to lock the objects.

3.1.1 Password Authorization:

Never use the password in the custom code.

Use the existing password authentication mechanism provided by the SAP NetWeaver platform, never implement your own

Never transmit the passwords in the plain text, if need be, to transmit passwords using the Secure Network Communications (SNC) protocol.

Avoid the administrator gaining access to the password. Use secure hash functions to prevent password recovery

The SAP Net Weaver platform uses secure hash values to store passwords.

Do not invent your own coding to encrypt the original password

Never record passwords in log /protocol /trace files. Do not use HTTP GET-requests since all parameters will be found in the URL. Use HTTP POST-requests instead. In general, you should avoid transmitting passwords, in particular with every request you send. Use secure mechanisms instead, such as digital certificates for example.

3.1.2 Secure Store and Forward Mechanism (SSF):

- Never use the custom code for storing and forwarding the secured data
- Never transmit the secured data with no encryption.

3.1.3 Audit Logs

- Logs should only be readable and never be changeable, due to traceability. Make sure, that there is a check implemented to deny unauthorized access to logs.
- Logs should not contain potentially confidential data such as credit card numbers or social security numbers. Instead, log such sensitive data in specially protected logs with authorization checks.

3.1.4 Web Dynpro & ABAP

Due to the architecture & Framework of Web Dynpro, SAP GUI automatic integration of output encoding, input validations etc. functionality is already available. Therefore development should always use Dynpro, SAP GUI, never to code outside Web Dynpro.

3.1.5 Get Method

Never use the 'GET' method to submit data, because the data is appended to the URL and may be easily manipulated. It is better to use the 'POST' method instead.

3.1.6 Decoding

Ensure, that the application does not repeat any character-decoding process. Decoding should be done by the operating system. Suppose, that the data remains encoded or contains unacceptable characters, treat the data as malicious one's and deny the input.

3.1.7 Cookies

- Do not store any cookies using the ABAP code. Cookie management needs to be done as per the guide lines. Do not store ANY data in a client cookie, unless you absolutely have to. Hackers can easily manipulate client-side cookies.
- Never create your own session management, use the SAP Net Weaver platform provides.

3.1.8 SQL checks

Never use dynamic program generation done with the ABAP key words , INSERT REPORT and GENERATE SUBROUTINE POOL

3.2 Techniques that Require Prior Approval

In order to simplify the ABAP Code that we will need to support and maintain going forward, please avoid the following list of ABAP commands and techniques if at all possible. Use of these commands/techniques requires prior approval by the SAP Development Manager, primarily because they are harder to debug:

3.2.1 Create new Function Group

- When creating a new custom Function module, or generation a table maintenance view, use the Function Group as specified in the Technical Specification. If no Function Group is explicitly mentioned in the Technical Specification, please consult with the Development Manager/Team Lead, who is responsible for creating and maintaining the Function Groups.
- Create a separate transport from any other transports so that it can be released through the landscape.

3.2.2 Import / Export to Memory

- Writing big blocks of data to memory may negatively impact performance, and exporting flags to memory make programs much more difficult to debug. Please consult with the Development Manager/Team Lead before using either the ABAP keywords import or Export to memory.
- When using EXPORT into Memory and IMPORT from memory, we must add comments in both the programs; where the memory would be imported and used in the called program and where the values are exported into Memory in the calling program.

3.2.3 Common Part

- Please consult with the Development Manager and other Team Members, as changing data declarations in common part section of a program will affect other programs.

3.2.4 Dynamic Form Calls

- Dynamic Form calls are to avoid; they make analyzing programs much more difficult, as it renders the “Where Used” function useless.

3.2.5 SQL

- Always use Open /Native SQL for ABAP techniques. For anything outside these, get the prior approval from development Manager.

3.2.6 File Management

- Accept the input files from either the SAP application server or from XI server. Development that involves the Files from any other location will need prior approval from Development manager.
- Don't implement file access functionality that is based on user input, unless there is no other alternative. When no alternative, get the prior approval from development manager.

3.3 Techniques to use sparingly

The following techniques should not be used more than necessary, in order to create code that is easy to read and debug:

3.3.1 Field Symbols

Field symbols can make programs extremely difficult to analyze and debug. Avoid wherever possible.

3.3.2 Get/Set

Only use Get/Set for initializing selection-screen content with user parameters, as using the Get/Set functionality to pass values between programs makes analysis and debugging more difficult.

3.3.3 Field Groups

Using several different field groups and summarization levels, when building an extract are to be avoided.

3.4 Smartforms Vs SAPscripts

New Forms will be developed using Smart Forms. Existing Forms will continue to use their existing design base unless otherwise dictated by the Business.

3.5 ABAP List Viewer (ALV)

Make sure all the list outputs should be compatible with the ALV functionality within SAP.

4. Error Handling

4.1.1 Always test SY-SUBRC

Every command that returns a return code must be followed by a test whether or not the return code is OK. Deciding which errors should cause the program to fail should be reviewed at the Code Review

4.1. Notification via Workflow:

If a program detects an error which does not need to be fixed immediately by Production Support but should be brought to someone's attention, use email notification – via workflow – to send messages to appropriate Functional and/or Development personnel.

4.2. Standard Error Reporting in BDC and Call Transaction

Inbound interface programs using either BDC or Call Transaction and must write error transactions to Error BDC sessions for correction and processing by the end user. BDC is **NOT** recommended and prior approval from the RICE COE is required. If BDC is the only solution, written document with all the options analyzed that should be saved in eProject enhancement folder.

4.3. BAPI errors

Always use BAPIRET2 parameter ('exporting' from the BAPI).

4.4. Function Modules

When calling a function, ensure that exceptions are either handled in the calling program, or the comment out the *exceptions* block altogether if you want the called function to handle the error scenarios. In the latter case, ensure that all exceptions are handled using the *message ... raising ...* technique.

5. Modularization

If a block of code is executed more than once, it should be placed in a subroutine at the bottom of the code. This makes the code more readable, requires less indentation, and is easier to debug since the debugger can jump through an entire subroutine. Also, when possible, parameters should be passed to and from subroutines to make the purpose easier to understand.

Modularization units like Forms and Function Modules should always be used where appropriate. Do not execute Forms from other programs; use Function Modules instead. Use Includes when code is shared across multiple programs (Like data declarations, or common subroutines to add an entry into BDC table). When modularization is used, the respective modularization units must always be documented and all parameter passing indicated and described – implicit as well as explicit.

5.1. Forms

For good Modularization, the decision of whether or not to execute a subroutine should be made before the subroutine is called.

For ex:

```
If n gt 0.
Perform check_amount
Endif.
```

A FORM should be used for large coding blocks, especially within IF statements. Common sections of code accessed from several points in an ABAP should also be put into Forms. When documenting a Form the following header should be used as the standard. To get this standard template type the 'PERFORM' statement with the intended module name and double click it, if this subroutine does not already exist the ABAP editor will insert this template at the end of the program.

```
*-----
* Form name:
*-----
* Description:
*
*-----
*           Name  Usage
* Parameters:      PAR01 Invoice Amount
*                  PAR02 Calculated Tax Amount is returned
*-----
FORM ... USING PAR01 CHANGING PAR02
...
ENDFORM.
```

When defining parameters use 'fp_' as prefix and always define them with a like or type statement.

5.1.1. Parameter Passing in Forms

5.1.1.1. Forms used in only one program:

It is *recommended* that you pass many (or all) of the parameters to a form using the TABLES, USING and CHANGING options – particularly Tables and Work areas. This makes it easier for someone reading the program to follow the data flow.

5.1.1.2. Common, reusable Forms:

Forms designed, to be used in a number of different programs (forms stored in a common include, for example) should be like Function Modules – all of their input and output should be explicit. They should contain no global data – including TABLES statements (instead: SELECT ... FROM table INTO work-area). Whenever possible use a TYPE or LIKE statement when specifying the formal parameters of a subroutine.

5.2 Dialog Modules

Use the Documentation Standards for **each individual module** in a Module Pool. *The Header for the Module Pool is the same as that for ABAP programs.*

5.3 Include Programs

A bulky and complex program can be separated into smaller logical sub-programs. The

main program (executable - Type 1) contains INCLUDE statements which incorporate the sub- components (programs of Type I). The header for Include programs is the same as the header for other ABAP programs.

```
***** START OF INCLUDES *****
INCLUDE /PWEAVER/MMM_XXXX_TOP.      "Top declaration
INCLUDE /PWEAVER/MMM_XXX_PAI       "PAI Modules
INCLUDE /PWEAVER/MMM_XXX_PBO       "PBO Modules
INCLUDE /PWEAVER/MMM_XXX_POV       "POV Modules
INCLUDE /PWEAVER/MMM_XXX_POH       "POH Modules
INCLUDE /PWEAVER/MMM_XXX_FORMS     "Subroutines
***** END OF INCLUDES *****
```

INCLUDE files can't define their own Text Elements - any Text Elements to which they refer must be defined in the main program which invokes the INCLUDE file. Therefore, if it is possible that an INCLUDE file may be invoked from more than one main program, constant text that is used within the INCLUDE should be defined with the CONSTANTS statement.

5.4 Function Modules

A Function Module works well when the same functionality is used across many programs. When defining a new Function Module, the documentation standards should be followed.

5.4.1 Error Handling

Please use the following code for errors detected in Function Modules:
MESSAGE ID mid TYPE type NUMBER nr WITH f1 ... f4 RAISING exception"
command.

(This form of the ABAP "MESSAGE" command is only possible within a function module or a method.)

If the calling program does not handle the exception itself, the message is output.

The import and export parameters of function modules should have names beginning with 'IM_' and 'EX_' respectively. They should be documented with brief descriptions of the fields and their typical contents. At the minimum, the documentation 'Short Text' should be completed for each parameter.

5.4.2 Release status of Function Modules

Do not use standard Function Modules that are not released in custom programs. 'Released on' date on the 'Attributes' tab of a Function Module should be populated. (If a Function Module is not released, the label of the above field will read as 'Not Released').

If you cannot find standard function module released for your purpose, please do research on SAP Service Marketplace if there is an alternate option or function. If nothing is found then we need to work with SAP and COE to get approval to use the function module or find an alternate option. SAP will not support usage of non-released function modules.

Also please note, copying the function module to Y-function module is not an option because if SAP modifies the original function module, we will have no way of knowing and going back to the copied version for making updates.

5.5 User Exits

Addition of enhancement points:

User exits are the only technique to be used in modifying SAP-delivered programs. User Exits are organized under **Projects** (CMOD).

User Exits can have a major impact on performance since they are called from standard SAP transactions. All new Exits or significant increases in existing Exits must be reviewed and approved by the RICE COE and the Integration Team Lead before being transported to the Testing System.

To limit these occurrences of missing include in the user exit and incorrect code in the USER EXIT, the following is the procedure:

- Do not put any code in ex. MV45AFZZ, instead only add an include statement and add all of your code in that new include program.
- When you add new include statement (user-exit) to MV45AFZZ and somebody already has locked the program, a new task will be created under the existing request under your User Id. Create the new include in that same task under the existing request. Request to have this sub-task released ASAP. As soon as it has been released, you can start editing your include. Create a brand new transport number for this include.
- When whoever is locking program MV45AFZZ gets the transport released, your include statement and "empty" include will be transport along. Once you are finished with your code, you can transport your code separately, as it has its hooks into MV45AFZZ.
- Even if you move an empty include, make sure to test the user exit with it to prove that your empty include will not impact the entire user exist. Certain user exits (ex. project YAM_ORD2) may require a single statement "Move an import structure to an export structure."
- If nobody was locking MV45AFZZ, so follow the same procedure: first get the request with MV45AFZZ and empty include released, then start editing the include program in a brand new request.
- Do not to use Check/Stop/Exit Statement as standalone in the user exit includes; but developers can use these statements within a processing block like form..endform/ loop..endloop etc., in the user exit includes.

If everybody follows this procedure, will never have MV45AFZZ blocked for extended time anymore, only add "INCLUDE statements" to it, and only transport it along with "empty" includes. The actual user-exit code will be transported in separate requests. This will avoid all problems related to missing code, locking MV45AFZZ, version control, sequencing imports into ARS and ARP.

It also suggested to restrict the access to creating and changing repository objects to ONLY objects that belong to the **Custom development classes**.

There are two ABAP roles defined for this purpose, to ensure the process is followed. All current ABAP users will still have access to create and modify ProcessWeaver code but will not be able to modify any SAP code, and that all other SAP users will still have the same access as they do currently on all clients. All users will be able to release the

TASK only of their transports.

1. Definition of the 'SUPER' ABAP role (ZHA:ABAPDEVELOPER)

These users will have access, as they can change modify SAP user exit and also the ProcessWeaver custom codes developed under the custom development class. The responsibility will be to create an 'include' (following the naming standards as YAAU_TOID, where AA stands for function process) in the User Exit with custom development class and will release the User Exit and an empty include task will request the transport releaser/approver to release the transport request.

2. Definition ABAP role (ZHA:ABAPDEVELOPER_ANY)

All the users will have access only to create/change developed under the ProcessWeaver custom development class. Any 'include' to be coded after the 'Super ABAP' role create the appropriately.

This can be accomplished via the following authorization object in the ABAP development role.

Authority-check object 'S_DEVELOP'

id 'DEVCLASS' field value YAERO*

id 'OBJTYPE' *

id 'OBJNAME' *

id 'P_GROUP' *

id 'ACTVT' field value '02' and '01'.

6. Performance

There are many areas in an ABAP program where small changes to the code can affect dramatic performance increase. Some of the most common are listed here. Every Developer is responsible for writing code that performs efficiently. Please run **SCID/SLIN/SE30** after the completion of the program. For the comprehensive list of the tuning guidelines.

6.1 Database Access

Database access should be carefully designed before writing the code; the most efficient access should be chosen.

6.1.1 Master Data and Configuration Data

1. When a program is based on transaction data, we recommend that you save relevant Master data in an internal table and only access the database upon failed lookup in the internal table.
2. We recommend that all configurations related data should be retrieved into internal tables at the appropriate place.

6.1.2 SELECT and SELECT SINGLE

When using the SELECT statement, study the key and always provide as much of the left-most part of the key as possible. If the entire key can be qualified, code a SELECT SINGLE not just a SELECT. If you are only interested in the first row or there is only one row to be returned, using SELECT SINGLE can increase performance by up to 3x.

Note: When select statements return more than one row and if you need only

one row use **up to 1 row**.

6.1.3 Nested SELECTs versus table views or JOINS

Performance of nested SELECT loops is often very poor in comparison to a join. To improve performance and reduce network load, you should consider creating a view in the data dictionary then using this view to select data, or using a JOIN in the FROM clause in your SELECT statement. Consult the Tech Team regarding database access design. (Before creating custom views, look for Standard SAP views in the data dictionary).

6.1.4 Row-Level Processing of a table

Selecting data into an internal table using an array fetch (**INTO TABLE itab**) versus a SELECT-ENDELECT loop will give at least a 2x performance improvement. After the data has been put into the internal data, then row-level processing can be done.

6.1.5 SELECT * versus Selecting individual fields

In general, use a SELECT statement specifying a list of fields instead of a SELECT * to reduce network traffic and improve performance. For tables with only a few fields the improvements may be minor, but many SAP tables contain more than 50 fields when the program needs only a few. In the latter case, the performance gains can be substantial.

6.1.6 FOR ALL ENTRIES IN

When using the FOR ALL ENTRIES IN table addition to the SELECT statement a check must be made before the SELECT to confirm that at least one entry exists in the source table. This must be performed because the effect of FOR ALL ENTRIES IN with an empty table is to read ALL entries in the target database table.

6.2 Internal Tables

Make use the 4.x internal table types (Standard, Sorted or Hashed) wherever appropriate.

6.2.1 Reading single records of internal tables

When reading a single record in an internal table, the READ TABLE WITH KEY is not a direct READ. This means that if the data is not sorted according to the key, the system must sequentially read the table. Therefore, SORT the table and use READ TABLE WITH KEY BINARY SEARCH for better performance.

6.2.2 The SORT statement

e.g. SORT xtab BY field1 field2.

Proper care should be taken, when sorting an internal table to set the BY clause as definitively as possible. In the example **xtab** is sorted only by **field1** (Primary) and **field2** (secondary), whereas the statement **SORT xtab** will sort the table by *all* fields. This is obviously much more resource intensive. The **SORT** statement should be used instead of **APPEND SORTED BY**.

6.2.3 Small Internal tables Vs. Complete Internal Tables

In general it is better to minimize the number of fields declared in an internal table. While it may be convenient to declare an internal table using the LIKE command, in most cases, programs will not use all fields in the SAP standard table.

6.3 Logic Structures

6.3.1 Decision trees

Decision trees, like **IF**- and **CASE** statements, should have the most frequently occurring conditions first since these statements are processed sequentially. In the same vein, logical expressions should have the most frequent false conditions *first* in an **AND** expression and the most frequent true conditions *last* in an **OR** expression.

6.3.2 CASE vs. Nested IFs

When testing fields "equal to" something, use a CASE statement rather than an IF/ELSEIF series. The CASE statement is better for two reasons: it is easier to read and after about five IFs the performance of the CASE is more efficient.

6.4 Fields, Structures etc.

6.4.1 Field assignment

In general, the **MOVE-CORRESPONDING** statement is more resource intensive than single **MOVE**'s when assigning values to fields. Use **MOVE strucA TO strucB** where the two structures are identical, **MOVE-CORRESPONDING** where the structures are similar and single **MOVE**'s, where they are not.

6.4.2 Field Conversion

Use the required CONVERSION EXITS to convert the fields to SAP format

6.4.3 Field Includes

If a structure exists it is preferable to use INCLUDE <STRUCTURE> rather than specifying all the fields

6.5 General Coding

6.5.1 Subroutine Usage

For good modularization, the decision of whether or not to execute a subroutine should be made before the subroutine is called.

6.5.2 Dead Code

Avoid leaving "dead" code in the program. Comment out (or delete) variables that are not referenced and code that is not executed. Use program --> check -> -extended program check to see a list of variables that are not referenced statically.

7. General Standards

7.1 *Pretty Printer*

Use PRETTY PRINTER before releasing a program for transport (to avoid someone using it later by mistake and creating versions that cannot be compared easily). On Settings for Pretty Printer, always check 'Indent'. Choose Keyword Uppercase radio button under 'Convert Upper-/Lowercase'

7.2 *Check*

Whenever a finished program is parsed by clicking the Check button it should be 'syntactically correct', meaning that no warnings or errors exist.

7.3 *SY- fields and Selection-Screen Parameters*

Never change the contents of a **SY-** field or a Selection-Screen field. Move these fields into regular global or local variables if you need to change the content.

7.4 *Field Data Validation*

7.4.1 *Selection Screen Parameters*

Input parameters will be validated within the ABAP program based on the definition of the data variable. Appropriate error messages will be displayed.

It is good to validate single values in the AT SELECTION SCREEN event; avoid extensive database access.

7.4.2 *Input Data Errors*

Inbound interface ABAP programs will format data as required by the fields specified for the Interface Transaction and will submit those transactions via BDC or Call Transaction. Validation of input data will be handled by the underlying logic of the SAP Transaction being called by the BDC or Call Transaction execution. Errors detected due to invalid data format or content must be passed to an Error BDC session for correction by the appropriate end user. No additional field data validation is required.

7.4.3 *Input Validation*

All external input should be validated: for field length, data type, field length, white list (to accept only known unproblematic characters). For Web Dynpro **Existence and length check, Canonicalization, Type check and Range check will be automatically performed by the framework.**

If users are allowed to enter a URL within the input field, do restrict the domain of the URL and permit only the selection of approved Extensions (such as HTMLB, XHTMLB and PHTMLB) that accept input parameters via HTTP requests URL's.

Use ENCODE attribute, for all BSP-

Note: For Web Dynpro ABAP:

Due to the architecture of Web Dynpro, automatic integration of output encoding functionality is available. Therefore, no further output encoding functionality has to be implemented manually.

7.5 Text Handling

Constant text, which is printed on the report, can be stored as Text symbols. The best way is to hard code constant text/literal in the code and gives the option of maintaining Text Elements for multilingual clients.

Example:

WRITE: / 'Total Quantity on hold :'(001).

If there is no text saved under 001 then the text between the single quotes is used.

The advantages of this method are readability and only the text element needs to be maintained for multilingual clients.

7.6 Messages

Declare the message class in the report statement: While it is possible to specify the message class each time a message is output, it is easier to specify it once in the report statement. You can still use a message in another class than the one defined in the report by adding the class in parentheses after the message.

Examples:

Report /PWEAVER/MMM_XXXX message-id /PWEAVER/MMM_XX.

Message e001.

Message e231.

If it is likely that a message will be used repeatedly, avoid using generic (placeholder) messages in programs (messages composed only of &'s).

If a message is unique we recommend a format like the statement below:

message i999(/PWEAVER/MMM_XX) with 'Insert successful'(m05).

This message will use the contents of text-m05 with 'with', if it exists for sy-langu, else it will use the text in quotes. This will provide ease of debugging and position the code to work well in multi-lingual environments. (See Text Handling, above)

Common Forms included in multiple programs should always use an explicit message class in each message statement.

Use long text where appropriate.

If your program is to output a specific message, which requires further explanation, create a new message and fill out the long text form. When the user clicks on the message the long text will be displayed.

Use the correct message type:

Type	Description	On-line action	Background action
I	Informational	Press ENTER to continue	No action. Message in Job Log.
W	Warning	Correction possible	No action. Message in Job Log.
E	Error	Correction required	Program terminated
A	Abend	Transaction terminated	Program terminated

S	Success	Message on subsequent screen	No action
---	---------	------------------------------	-----------

7.7 'Extended Program Check' for ABAP Programs

The extended program check makes static checks that are too laborious for the normal syntax check. To access from the ABAP Editor: Program -> Check -> Extended program check.

The extended program check is a good way to double-check a program and cleanup unused variables and forms. It is *recommended* that you spend some time going through the extended check after writing a new program. Use Tcode SCID and SLIN for checking the programs.

Add code inspector checks: Object→Check→Code Inspector

ObjectCheckCode Inspector

7.7.1 Using the checks

For performance reasons, and to make them easier to use, the extended check is divided into 15 separate checks. Use F1 help to display details about a particular check. You can use any number of the sub-checks in any combination. The results are displayed as an overview (unless you have selected the first of the "Additional functions". If you double-click a list item, the error description appears. A second double-click branches to the point in the source code where the error occurred.

7.7.2 Extended Checks in ABAP Code Review

In the ABAP Code review you may be asked to address 'Error' items in the Extended Program Check **except**:

- Character strings
- Portability ASCII/EBCDIC

7.8 Report formatting

Blank lines are to be specified using 'SKIP <n>' as opposed to multiple 'WRITE /' statements. Use NUMBERED-TEXT for all screen text to be displayed. This is defined via include text and is language independent.

7.9 Batch data input

Batch data input sessions should be limited as approx. 250 transactions per BDC group. This number can vary greatly depending on the quality of the data being processed. Larger batch ABAP programs are to be avoided due to single-threaded nature of the batch environment. These programs are to be broken up into smaller batch programs, where appropriate.

Calling the existing function module, check for the release date and make sure use only the released function module. Please do not use the obsolete syntax.

Ex. GUI_upload instead use WS_upload

7.10 Lock objects

Lock objects should be created as function modules (Enqueue / Dequeue) in a customer function group.

7.11 Debugging

All debugging code must be removed from a program before migration to a controlled environment. This includes breakpoints and any code associated with testing.

8. Additional Naming Standards

8.1 Other Program Objects

Program naming standards are listed in 2.2 above section.

8.1.1 ABAP Module Pools

The naming standard for a program that calls a series of screens is the same as for any other program.

8.1.2 Temporary ABAP objects

Temporary ABAP objects must begin with a ZZ. These temporary ABAP objects must be saved as local objects and hence they cannot be transported.

Example: ZZOLE – test program to check OLE with EXCEL

8.1.3 Program Attributes

Every ABAP program has a set of attributes that describes and classifies it. Maintaining these correctly allows for ease of use and easy location of Programs.

Title Required. Be descriptive.

Type Required – use the pull-down menu.

Status Required – use the pull-down menu.

Application Use pull-down menu to select appropriate SAP Application.

Authorization Group Program name, or Functional Process

8.2 Object Oriented Objects

8.2.1 Classes and Interfaces

Classes and Interfaces use the same naming standards as Programs; only the 'Function' code is two characters rather than one:

Position 1-9 /pweaver/

Position 10-11 CL

Position 12 – (underscore)

Position 13 and after Descriptive. Sequential numbering is permitted only if it follows at least three characters of description or abbreviation.

8.3 Corrections/Transports process

Assigning standard naming conventions to corrections, repairs, and transports requests ensure that they are easy to find, etc. All custom repository objects will be triggered from

development server of the client.

8.3.1 Text on Corrections/Transports

1 st – 2 nd position	PW
3 rd position	_ (underscore)
4 th – 6 th position	Module name (ECS, PPS, EFA Etc...)
7 th position	_ (underscore)
8 th position onwards	description of the object being developed

Example: PW_ECS_initial transport

8.4 Data Dictionary Objects

8.4.1 Tables, Table Types, Structures and Views

Use the /pweaver/ as first nine characters and remaining see

8.4.1.1 Data Dictionary Table Fields

Field names in Data Dictionary Tables are freely definable only in case of custom table. In case of standard SAP table append/include, use “ZZXXXXXX” for the field names.

Where possible, use the same name as the data element associated with the field. If multiple fields in the table use the same data element, these fields must be named in a meaningful manner.

Where possible, an existing SAP data element must be used. If that is not possible, a new data element (and possibly a new domain) must be created using the standards outlined in this document.

8.4.1.2 Data Elements and Domains

Use existing SAP Data Elements and Domains wherever possible.

Custom Data Element and Domain names should always begin with ‘/pweaver/de_’ (since SAP uses Data Elements and Domains beginning with “). Remaining characters should be descriptive.

e.g. /pweaver/de_carrier

If you need a custom field description, you will need to create a custom Data Element.

If you need to create a custom Values list, you will need to create a custom Domain and a custom Data Element.

Where possible, use the same name for the domain and the data element.

8.4.2 Type Groups & Type Pools

The maximum length of a type group/type pool name is 5 characters.

e.g. ZFCMS

8.4.3 Search Help and Match code Objects

Use Search Help, not Match code Objects.

Search Help names: use the same first nine characters that are described for Program Names, the next three characters are ‘sh_’ and remaining characters

are description.

8.4.4 Lock Objects

Setting and removing locks use lock Objects to synchronize access to data objects. The Lock Object is defined in the Dictionary and will automatically create two ABAP Function Modules (Enqueue and Dequeue Routines) to be used in Dialog programming.

- The first two characters will be 'EZ' (Custom Lock Object)
- Remaining characters should contain the name of the table being locked. If the table name starts with a 'Z' that 'Z' can be the second character.
- The Underscore character '_' is allowed
e.g. **EZL_KNA1**

8.5 Program Variants

- Variant names should be descriptive
- Never use your own name like John1, John2, etc.
- It is recommended to create variants in Development server and transport them to Production.
- System Variants always start with 'CUS&' or 'Z'

8.6 Transaction Codes

Position 1-9	/pweaver/
Position 10-12	Module name (ECS, PPS, EFA Etc...)
Remaining positions:	Description
e.g.	/pweaver/ecs_xxxxxxxxx

8.7 SAP Script Objects

8.7.1 Layout Sets (Forms)

The Layout Sets should have the following naming convention

Position 1-9	/pweaver/
Position 10-11	SC
Position 12	_(Underscore)
Remaining positions:	Description

8.7.2 Style

Custom styles (SE72):

Position 1-9	/pweaver/
Positions 10-11	SS
Position 12	_(Underscore)
Remaining positions:	Description

8.7.3 Paragraphs and Character Formats

If a Layout Set is totally custom, Paragraph and Character Format names should be entirely descriptive.

If a Layout Set is copied from an SAP Layout Set and you then create some custom Paragraphs and/or Character Formats, begin your custom names with 'Y'

